



THE VIBECODER PLAYBOOK • AI PROMPT PACK

# The Vibecoder Playbook

Guía Paso a Paso & Prompts Expertos para Construir y Desplegar APIs con IA en Minutos

Cursor AI (.cursorrules)

Windsurf

Claude 3.7 / 3.5 Sonnet

ChatGPT (GPT-4o / o3)

GitHub Copilot

Antigravity

Convierte cualquier idea en un backend listo para producción. Incluye el **Prompt Maestro de Sistema**, generadores de modelos de datos, creadores de features/CRUDs en slices verticales, protección con roles y autenticación JWT, tests automatizados y guía de despliegue en **Render / Railway + Neon PostgreSQL**.

**Starter Kit:** Enterprise .NET 10 & C# 13

**Patrón:** Clean Architecture + CQRS + MediatR

**Edición:** Vibecoder Masterclass v1.0

**Estado:** 100% Copy-Paste Ready

## ⚡ ¿Qué es esta Guía y Cómo Usarla?

---

Como **Vibecoder**, tu ventaja competitiva es la velocidad. Sin embargo, las IAs suelen cometer errores críticos cuando programan backends desde cero: crean controladores gigantescos, mezclan lógica de base de datos con APIs, olvidan la expiración de tokens o escriben código frágil.

### 💡 La Solución Definitiva

Este **Starter Kit Enterprise** ya tiene resuelto el 90% del trabajo pesado de ingeniería (Clean Architecture, JWT con Refresh Tokens, PostgreSQL 17, Scalar API Docs, Docker y Seguridad OWASP). Con estos prompts, tu IA generará código que encaja como un guante en la arquitectura sin romper nada.



# Prompt Maestro (Instrucciones de Sistema / .cursorrules)

Copia y pega este bloque en tu archivo `.cursorrules`, en las reglas de proyecto de Windsurf o al inicio de cualquier conversación con tu IA:

## PROMPT MAESTRO DE SISTEMA (.CURSORRULES)

Copiar y Pegar en la IA

Eres un Arquitecto de Software Senior especializado en .NET 10, C# 13 y Clean Architecture. Estamos trabajando sobre una base de código estructurada con Clean Architecture y CQRS (MediatR).

### REGLAS ERICTAS QUE DEBES CUMPLIR SIEMPRE:

#### 1. ESTRUCTURA DE CAPAS:

- BackendTemplate.Domain: Entidades puras (heredando de AuditableEntity y ISoftDeletable), Result<T>, Error, Enums. Cero dependencias externas.
- BackendTemplate.Application: Casos de uso (Features/NombreEntidad/Commands o Queries). Interfaces (IApplicationDbContext, IIdentityService), validadores FluentValidation, pipeline behaviors.
- BackendTemplate.Infrastructure: Implementación de EF Core, PostgreSQL (Npgsql), Identity, interceptores y configuraciones Fluent API.
- BackendTemplate.Api: Controladores heredando de ApiControllerBase, middlewares, Program.cs. No pongas lógica de negocio en los controladores.

#### 2. PATRÓN RESULT & ERRORES:

- Todos los Handlers deben retornar Result o Result<T>.
- Usa Error.NotFound(), Error.Validation(), Error.Conflict(), Error.Unauthorized(), Error.Forbidden() para fallos semánticos. No arrojes excepciones para flujos normales de negocio.

#### 3. VALIDACIONES:

- Cada Command debe tener su clase Validator heredando de AbstractValidator<TCommand>. El ValidationBehavior de MediatR los ejecuta automáticamente.

#### 4. PERSISTENCIA:

- Las entidades se auditan automáticamente (CreatedAt, CreatedBy, LastModifiedAt) y se borran con Soft-Delete vía AuditableEntitySaveChangesInterceptor.
- En consultas de solo lectura usa .AsNoTracking().

#### 5. API Y DOCUMENTACIÓN:

- Los controladores heredan de ApiControllerBase y retornan HandleResult(await Mediator.Send(...)).
- La documentación interactiva se genera automáticamente con Scalar en /scalar/v1.

Cuando te pida una nueva feature, genera siempre el código completo para todas las capas necesarias sin omitir ninguna.



## Fase 1: De tu Idea a la Base de Datos

Usa este prompt cuando tengas una idea de negocio y quieras que la IA genere el modelo relacional exacto para el Starter Kit.

### PROMPT 1: MODELADO DE BASE DE DATOS

Copiar y Pegar en la IA

Tengo la siguiente idea de negocio para mi aplicación:

"[DESCRIBE TU IDEA AQUÍ, EJ: Una plataforma SaaS de reservas para consultorios médicos con Doctores, Pacientes, Citas y Pagos]"

Actúa como Arquitecto de Datos .NET y genera el modelo de entidades para el Enterprise Starter Kit:

1. Diseña las entidades de dominio necesarias en C# 13 que hereden de ``AuditableEntity`` e implementen ``ISoftDeletable``.
2. Incluye métodos de negocio (ej. `CancelarCita`, `CompletarPago`) y un constructor claro con encapsulación (``private set``).
3. Agrega las propiedades ``DbSet<T>`` necesarias para la interfaz ``IApplicationDbContext`` en `Application`.
4. Genera las clases de configuración Fluent API en `Infrastructure` (``EntityTypeConfiguration<T>``) con tipos de columna PostgreSQL, índices únicos y relaciones (`HasOne/HasMany`).
5. Dame el código listo para copiar y pegar archivo por archivo.

## ⚡ Fase 2: Creación de Módulos & CRUD Completo

Usa este prompt para generar una funcionalidad completa en vertical slices (Command, Query, Validator, Handler, Fluent API y Controller).

### PROMPT 2: GENERADOR DE FEATURES & CRUD

Copiar y Pegar en la IA

Quiero agregar el módulo/feature "[NOMBRE\_DEL\_MODULO, EJ: Appointments / Citas]" a mi backend.

La entidad tiene los siguientes campos:

- DoctorId (Guid, requerido)
- PatientId (Guid, requerido)
- AppointmentDate (DateTime, requerida)
- Reason (string, máx 250 caracteres)
- Status (Enum: Scheduled, Completed, Cancelled)

Genera el código completo siguiendo estrictamente la arquitectura del Starter Kit:

1. DOMAIN: Entidad de dominio con métodos de negocio y soft-delete.
2. APPLICATION:
  - CreateAppointmentCommand + Validator + Handler
  - UpdateAppointmentCommand + Validator + Handler
  - DeleteAppointmentCommand + Validator + Handler (Soft-delete)
  - GetAppointmentByIdQuery + Handler + DTO
  - GetAppointmentsWithPaginationQuery + Handler (usando PaginatedResult<T>)
3. INFRASTRUCTURE:
  - Configuración Fluent API con HasQueryFilter(!IsDeleted) e índices.
4. API:
  - AppointmentsController heredando de ApiControllerBase con endpoints REST bien nombrados y Swagger/Scalar annotations.

Escribe el código completo, sin placeholders de tipo "// todo" ni código dummy.



## Fase 3: Seguridad, Autenticación y Roles

Aprende a proteger cualquier endpoint y obtener los datos del usuario conectado.

### PROMPT 3: PROTECCIÓN CON JWT Y ROLES

[Copiar y Pegar en la IA](#)

Necesito proteger los endpoints de mi controlador `[NOMBRE\_CONTROLADOR, EJ: InvoicesController]` y vincular los datos al usuario que inició sesión.

Requisitos:

1. Solo usuarios autenticados con JWT pueden acceder (`[Authorize]`).
2. El endpoint de eliminación o modificación administrativa debe requerir el rol `Administrator` (`[Authorize(Roles = "Administrator")]`).
3. En el comando de creación, el ID del usuario creador debe obtenerse de `ICurrentUserService.UserId` y no del cuerpo de la petición (por seguridad).
4. En las consultas (Queries), si el usuario es un `User` estándar, solo debe ver sus propios registros; si es `Administrator`, puede ver todos los registros.

Muestra cómo modificar el Controller y el QueryHandler de Application para implementar estas reglas.

## Fase 4: Testing Automatizado

Genera pruebas unitarias e integradas para garantizar que tus módulos nunca se rompan en producción.

### PROMPT 4: SUITE DE PRUEBAS AUTOMATIZADAS

Copiar y Pegar en la IA

Genera la suite de pruebas automatizadas para el módulo `[NOMBRE\_DEL\_MODULO, EJ: Invoices]`:

#### 1. TESTS UNITARIOS (en BackendTemplate.UnitTests):

- Pruebas para el CommandValidator (probando casos válidos e inválidos como campos vacíos, valores negativos o longitudes excedidas).
- Pruebas para los Handlers usando `TestDbContextFactory.Create()` (InMemory DB) verificando que devuelvan `Result.Success` o `Result.Failure(Error.NotFound)`.

#### 2. TESTS DE INTEGRACIÓN (en BackendTemplate.IntegrationTests):

- Prueba HTTP usando `CustomWebApplicationFactory` y la colección `[Collection("IntegrationTests")]`.
- Verifica que un usuario no autenticado reciba 401 Unauthorized.
- Verifica que con un payload válido devuelva 200 OK y la estructura `ApiResponse<T>`.

Escribe las pruebas listas para ejecutarse con `dotnet test`.



## Fase 5: Despliegue en 5 Minutos (Render + Neon)

### Guía Rápida de Despliegue:

1. **Crear BD en Neon.tech:** Crea un proyecto gratuito en [Neon.tech](https://neon.tech) y copia el Connection String.
2. **Crear Web Service en Render.com:** Conecta tu repositorio de GitHub, selecciona el entorno **Docker** y Render usará el `Dockerfile` optimizado.
3. **Configurar Variables de Entorno en Render:**

#### ⚙️ VARIABLES DE ENTORNO PARA RENDER / RAILWAY

```
ASPNETCORE_ENVIRONMENT=Production
ASPNETCORE_HTTP_PORTS=8080
ConnectionStrings__DefaultConnection=Host=tu-
host.neon.tech;Database=neondb;Username=tu_user;Password=tu_pass;SSL Mode=Require;Trust Server
Certificate=true;
JwtSettings__Secret=MiclaveSuperSecretaYExtremadamenteLargaDeMasDe64CaracteresSeguros123!
JwtSettings__Issuer=https://tu-api.onrender.com
JwtSettings__Audience=https://tu-frontend.vercel.app
JwtSettings__ExpirationMinutes=60
JwtSettings__RefreshTokenExpirationDays=7
CorsSettings__AllowedOrigins__0=https://tu-frontend.vercel.app
Database__ApplyMigrationsOnStartup=true
```



#### ¡Auto-Migración y Auto-Seed!

Con `Database__ApplyMigrationsOnStartup=true`, la API creará automáticamente las tablas en Neon y sembrará el usuario `admin@template.com` con contraseña `Admin123!` en el primer arranque.



## Atajos y Referencia Rápida

---

| Acción                            | Comando / Atajo                                                    |
|-----------------------------------|--------------------------------------------------------------------|
| Iniciar Base de Datos Local       | <code>docker compose up -d postgres redis mailpit</code>           |
| Ejecutar API Localmente           | <code>dotnet run --project BackendTemplate.Api</code>              |
| Documentación Interactiva Scalar  | Navega a <code>http://localhost:5000/scalar/v1</code>              |
| Ver Emails Locales (Mailpit)      | Navega a <code>http://localhost:8025</code>                        |
| Ejecutar Toda la Suite de Pruebas | <code>dotnet test</code>                                           |
| Probar Peticiones HTTP            | Abre <code>backend-requests.http</code> en VS Code / Visual Studio |